

Phase 2 Optimization Derivatives

Perimeter and Area Quantities Implemented in `surface-partition`

May 2026

Abstract

This document derives every quantity that the Phase 2 perimeter-refinement optimizer evaluates during an IPOPT iteration: the perimeter objective $P(\boldsymbol{\lambda})$, its gradient ∇P , and its Hessian $\nabla^2 P$; the area constraint values $A_k(\boldsymbol{\lambda})$, their Jacobian J , and the multiplier-weighted Hessian $\sum_k \mu_k \nabla^2 A_k$. Quantities belonging to *regular* boundary segments and triangles are derived in closed form. Quantities involving *Steiner* / *triple-point* geometry are computed by finite differences; the FD schemes are described precisely and their limitations noted. All formulas are cross-referenced to the implementing Python functions in `src/partition/`.

Contents

1	Overview	3
2	Notation and Setup	3
2.1	The mesh	3
2.2	Variable Points and the λ convention	3
2.3	The edge direction identity	3
3	Regular Perimeter	4
3.1	Perimeter value	4
3.2	Perimeter gradient	4
3.3	Perimeter Hessian	5
4	Regular Cell Area	5
4.1	1-inside case	6
4.1.1	Area value	6
4.1.2	Area Jacobian	6
4.1.3	Area Hessian	6
4.2	2-inside case	7
4.2.1	Area value	7
4.2.2	Area Jacobian	7
4.2.3	Area Hessian	7
5	Steiner / Triple-Point Contributions	8
5.1	The Fermat–Torricelli point (forward formula)	8
5.2	Steiner perimeter value	9
5.3	Steiner area value	9
5.4	Steiner derivatives: finite-difference schemes	9
5.4.1	Perimeter gradient (forward FD)	10
5.4.2	Area Jacobian (forward FD, sparse output)	10

5.4.3	Perimeter Hessian (nested central FD)	10
5.4.4	Area Hessian (nested forward FD)	10
5.4.5	Precision limitation	11
6	Lagrangian Hessian Assembly	11
6.1	The Lagrangian	11
6.2	Hessian assembly	11
6.3	Lower-triangle convention	12
6.4	Sparsity structure	12

1 Overview

Phase 2 of the **surface-partition** pipeline solves the constrained nonlinear program

$$\min_{\boldsymbol{\lambda} \in \mathbb{R}^n} P(\boldsymbol{\lambda}) \quad \text{subject to} \quad A_k(\boldsymbol{\lambda}) = \bar{A}, \quad k = 0, \dots, N-2, \quad (1)$$

where

- $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_n)^\top$ collects the scalar parameters of all active Variable Points (VPs),
- $P(\boldsymbol{\lambda})$ is the total boundary perimeter,
- $A_k(\boldsymbol{\lambda})$ is the surface area of cell k , and
- $\bar{A} = A_{\text{total}}/N$ is the equal-area target.

The solver used is IPOPT [2] (Interior-Point OPTimizer). At each iteration IPOPT requires the Lagrangian Hessian

$$H = \sigma \nabla^2 P + \sum_{k=0}^{N-2} \mu_k \nabla^2 A_k, \quad (2)$$

in addition to gradients and Jacobians. The scalar σ (objective scaling factor) and the Lagrange multipliers μ_k are supplied by IPOPT at each callback.

Structure of this document. Sections 2–4 cover the regular (non-Steiner) path, which is fully analytical. Section 5 covers the Steiner / triple-point path, where the forward values are analytical but *all derivatives are currently computed by finite differences*. Section 6 describes how the pieces assemble into the Lagrangian Hessian and its sparsity pattern.

2 Notation and Setup

2.1 The mesh

The surface is represented as a triangulated mesh $(\mathbf{V}, \mathbf{F})$, where $\mathbf{V} \in \mathbb{R}^{M \times 3}$ is the vertex position matrix and $\mathbf{F}$ is the face array. The mesh lives in $\mathbb{R}^3$; all cross-product formulas are three-dimensional unless otherwise noted.

2.2 Variable Points and the λ convention

A *Variable Point* (VP) i sits on a mesh edge $(v_{1,i}, v_{2,i})$ with the convention $v_{1,i} < v_{2,i}$ (smaller index first). Its three-dimensional position is the affine combination

$$\mathbf{p}_i(\lambda_i) := \lambda_i \mathbf{V}[v_{1,i}] + (1 - \lambda_i) \mathbf{V}[v_{2,i}], \quad \lambda_i \in [0, 1]. \quad (3)$$

- $\lambda_i = 1$: VP is at the vertex with the *smaller* index.
- $\lambda_i = 0$: VP is at the vertex with the *larger* index.

Code: VP position

```
vectorized_perimeter.py: _compute_vp_positions(pa)
p[i] = pa.vp_lambda[i] * V[pa.vp_edge.v1[i]]
      + (1 - pa.vp_lambda[i]) * V[pa.vp_edge.v2[i]]
```

2.3 The edge direction identity

Define the *edge direction* of VP i :

$$\mathbf{d}_i := \mathbf{V}[v_{1,i}] - \mathbf{V}[v_{2,i}] \in \mathbb{R}^3. \quad (4)$$

Differentiating (3) with respect to λ_i gives the fundamental identity used throughout all derivative calculations:

$$\boxed{\frac{\partial \mathbf{p}_i}{\partial \lambda_i} = \mathbf{d}_i, \quad \frac{\partial^2 \mathbf{p}_i}{\partial \lambda_i \partial \lambda_i} = \mathbf{0}.} \quad (5)$$

The position is *affine* in λ_i , so all second derivatives of $\mathbf{p}_i$ vanish. This dramatically simplifies the Hessian calculations in Sections 3 and 4: every second-order term that would involve $\partial^2 \mathbf{p}_i / \partial \lambda_i^2$ is zero.

3 Regular Perimeter

3.1 Perimeter value

The boundary between cells is represented as a collection of *segments*, each connecting two VPs. Let segment s connect VP a and VP b , and let w_s be its multiplicity weight:

$$w_s = \begin{cases} 2 & \text{if the segment separates two distinct cells (cell}_a \neq \text{cell}_b), \\ 1 & \text{otherwise (fallback: triple-point void segment, cell}_a = \text{cell}_b). \end{cases} \quad (6)$$

The weight $w_s = 2$ reflects the double-counting convention: each proper inter-cell boundary edge is shared by exactly two cells and therefore counted once in each cell's perimeter. The total regular perimeter is

$$P_{\text{reg}}(\boldsymbol{\lambda}) = \sum_s w_s \|\mathbf{p}_{a(s)} - \mathbf{p}_{b(s)}\|. \quad (7)$$

Code: regular perimeter value

```
vectorized_perimeter.py: compute_total_perimeter(pa)
```

3.2 Perimeter gradient

For a single segment s with endpoints $\mathbf{p}_a$ and $\mathbf{p}_b$, define

$$\boldsymbol{\Delta}_s := \mathbf{p}_a - \mathbf{p}_b, \quad r_s := \|\boldsymbol{\Delta}_s\|, \quad \hat{\boldsymbol{\Delta}}_s := \boldsymbol{\Delta}_s / r_s. \quad (8)$$

The contribution of s to the perimeter is $L_s = w_s r_s$. Applying the chain rule with identity (5):

$$\frac{\partial L_s}{\partial \lambda_a} = w_s \frac{\boldsymbol{\Delta}_s \cdot \mathbf{d}_a}{r_s}, \quad (9)$$

$$\frac{\partial L_s}{\partial \lambda_b} = w_s \frac{-\boldsymbol{\Delta}_s \cdot \mathbf{d}_b}{r_s}. \quad (10)$$

The sign difference arises because $\partial \boldsymbol{\Delta}_s / \partial \lambda_a = \mathbf{d}_a$ but $\partial \boldsymbol{\Delta}_s / \partial \lambda_b = -\mathbf{d}_b$.

The total gradient accumulates over all segments:

$$\frac{\partial P_{\text{reg}}}{\partial \lambda_i} = \sum_{s: a(s)=i} w_s \frac{\boldsymbol{\Delta}_s \cdot \mathbf{d}_i}{r_s} + \sum_{s: b(s)=i} w_s \frac{-\boldsymbol{\Delta}_s \cdot \mathbf{d}_i}{r_s}. \quad (11)$$

Code: perimeter gradient

```
vectorized_perimeter.py: compute_perimeter_gradient(pa)
grad_vp1[s] = sum(diff * dv1, axis=1) / lengths[s]
grad_vp2[s] = sum(-diff * dv2, axis=1) / lengths[s]
```

3.3 Perimeter Hessian

We compute $\partial^2 L_s / \partial \lambda_i \partial \lambda_j$ for the same single segment. The key tool is the Hessian of the Euclidean norm $g(\Delta) = \|\Delta\|$:

$$\frac{\partial^2 \|\Delta\|}{\partial \Delta_m \partial \Delta_n} = \frac{\delta_{mn}}{r} - \frac{\Delta_m \Delta_n}{r^3} = \frac{1}{r} (\mathbf{I} - \hat{\Delta} \hat{\Delta}^\top)_{mn}, \quad (12)$$

where $(\mathbf{I} - \hat{\Delta} \hat{\Delta}^\top) = \mathbf{I} - \hat{\Delta} \hat{\Delta}^\top$ is the projection onto the subspace perpendicular to Δ .

Since $\partial \Delta / \partial \lambda_a = \mathbf{d}_a$ and $\partial \Delta / \partial \lambda_b = -\mathbf{d}_b$, the chain rule gives:

$$H_{aa}^{(s)} = w_s \mathbf{d}_a^\top \frac{(\mathbf{I} - \hat{\Delta}_s \hat{\Delta}_s^\top)}{r_s} \mathbf{d}_a = \frac{w_s}{r_s} \left(\|\mathbf{d}_a\|^2 - (\mathbf{d}_a \cdot \hat{\Delta}_s)^2 \right), \quad (13)$$

$$H_{bb}^{(s)} = w_s \mathbf{d}_b^\top \frac{(\mathbf{I} - \hat{\Delta}_s \hat{\Delta}_s^\top)}{r_s} \mathbf{d}_b = \frac{w_s}{r_s} \left(\|\mathbf{d}_b\|^2 - (\mathbf{d}_b \cdot \hat{\Delta}_s)^2 \right), \quad (14)$$

$$H_{ab}^{(s)} = w_s \mathbf{d}_a^\top \frac{(\mathbf{I} - \hat{\Delta}_s \hat{\Delta}_s^\top)}{r_s} (-\mathbf{d}_b) = -\frac{w_s}{r_s} \left(\mathbf{d}_a \cdot \mathbf{d}_b - (\mathbf{d}_a \cdot \hat{\Delta}_s)(\mathbf{d}_b \cdot \hat{\Delta}_s) \right). \quad (15)$$

Remark 3.1. The factor $(\mathbf{I} - \hat{\Delta}_s \hat{\Delta}_s^\top)/r_s$ is the projection perpendicular to the segment direction, divided by the segment length. When $\mathbf{d}_i$ is *parallel* to Δ_s , the entry is zero — moving along the segment does not change its length to first order, which the gradient already captures. When $\mathbf{d}_i$ is *perpendicular*, the entry equals $w_s \|\mathbf{d}_i\|^2 / r_s$ (maximum curvature).

The full perimeter Hessian is the sum over all segments:

$$\frac{\partial^2 P_{\text{reg}}}{\partial \lambda_i \partial \lambda_j} = \sum_s H_{ij}^{(s)} \mathbf{1}[\{i, j\} \subseteq \{a(s), b(s)\}]. \quad (16)$$

Only pairs (i, j) that are both endpoints of a common segment contribute; the pattern is determined at compile time from `pa.seg_vp1` and `pa.seg_vp2`.

Code: perimeter Hessian

```
vectorized_perimeter.py: compute_perimeter_hessian_sparse(pa)
H_aa = w * (da_dot_da - da_dot_dh**2) / r
H_bb = w * (db_dot_db - db_dot_dh**2) / r
H_ab = -w * (da_dot_db - da_dot_dh * db_dot_dh) / r
Accumulated with np.add.at into the pre-allocated sparse values array at offsets
pa.seg_hess_off_aa, pa.seg_hess_off_bb, pa.seg_hess_off_ab.
```

4 Regular Cell Area

A mesh triangle T is a *boundary triangle* if the cell boundary crosses it — that is, if its three vertices do not all belong to the same cell. Each boundary triangle contributes to the area of exactly one cell. There are two structural cases depending on how many of the triangle’s mesh vertices lie *inside* the cell (Fig. 1).

Code: area arrays

```
Arrays pa.btri_n_inside, pa.btri_v_in, pa.btri_vp1, pa.btri_vp2, pa.btri_cell fully characterize each boundary triangle.
```

<i>1-inside:</i>	<i>2-inside:</i>
Triangle $T = (v_{\text{in}}, v_{\text{out1}}, v_{\text{out2}})$.	Triangle $T = (v_{\text{in1}}, v_{\text{in2}}, v_{\text{out}})$.
VPs: pc_1 on edge $(v_{\text{in}}, v_{\text{out1}})$, pc_2 on edge $(v_{\text{in}}, v_{\text{out2}})$.	VPs: pc_1 on edge $(v_{\text{in1}}, v_{\text{out}})$, pc_2 on edge $(v_{\text{in2}}, v_{\text{out}})$.
Cell fragment = $\triangle(p_{\text{in}}, \text{pc}_1, \text{pc}_2)$.	Cell fragment = $\text{quad}(p_{\text{in1}}, \text{pc}_1, \text{pc}_2, p_{\text{in2}})$.

Figure 1: The two boundary-triangle cases. Filled circles mark vertices inside the cell; open circles mark vertices outside. Stars mark Variable Points (VPs) on the crossing edges.

4.1 1-inside case

Let $p_{\text{in}} = \mathbf{V}[v_{\text{in}}]$ (fixed mesh vertex), $\text{pc}_1 = \mathbf{p}_1(\lambda_1)$, $\text{pc}_2 = \mathbf{p}_2(\lambda_2)$. Define

$$\mathbf{u} := \text{pc}_1 - p_{\text{in}}, \quad \mathbf{v} := \text{pc}_2 - p_{\text{in}}. \quad (17)$$

4.1.1 Area value

$$A = \frac{1}{2} \|\mathbf{u} \times \mathbf{v}\|. \quad (18)$$

Let $\mathbf{C} := \mathbf{u} \times \mathbf{v}$, $\|\mathbf{C}\| = 2A$, and $\hat{\mathbf{n}} := \mathbf{C} / \|\mathbf{C}\|$.

4.1.2 Area Jacobian

Differentiating (18) with respect to λ_1 and λ_2 , and using $\partial \mathbf{u} / \partial \lambda_1 = \mathbf{d}_1$, $\partial \mathbf{v} / \partial \lambda_1 = \mathbf{0}$, $\partial \mathbf{u} / \partial \lambda_2 = \mathbf{0}$, $\partial \mathbf{v} / \partial \lambda_2 = \mathbf{d}_2$:

$$\frac{\partial A}{\partial \lambda_1} = \frac{1}{2} \hat{\mathbf{n}} \cdot (\mathbf{d}_1 \times \mathbf{v}), \quad \frac{\partial A}{\partial \lambda_2} = \frac{1}{2} \hat{\mathbf{n}} \cdot (\mathbf{u} \times \mathbf{d}_2). \quad (19)$$

Code: 1-inside Jacobian

```
vectorized_area.py: compute_area_jacobian_analytical, 1-inside branch:
dA_d11 = 0.5 * np.sum(n_hat * np.cross(d1, v), axis=1)
dA_d12 = 0.5 * np.sum(n_hat * np.cross(u, d2), axis=1)
```

4.1.3 Area Hessian

Introduce the auxiliary cross products

$$\mathbf{g}_1 := \mathbf{d}_1 \times \mathbf{v} = \frac{\partial \mathbf{C}}{\partial \lambda_1}, \quad \mathbf{g}_2 := \mathbf{u} \times \mathbf{d}_2 = \frac{\partial \mathbf{C}}{\partial \lambda_2}. \quad (20)$$

Differentiating the unit-normal identity $A = \frac{1}{2} \|\mathbf{C}\|$ a second time, using the standard formula for the derivative of a unit vector, and noting that $\partial^2 \mathbf{C} / \partial \lambda_1^2 = \partial^2 \mathbf{C} / \partial \lambda_2^2 = \mathbf{0}$ (because $\partial^2 \mathbf{p}_i / \partial \lambda_i^2 = \mathbf{0}$):

$$\frac{\partial^2 A}{\partial \lambda_1 \partial \lambda_1} = \frac{\|\mathbf{g}_1\|^2 - (\hat{\mathbf{n}} \cdot \mathbf{g}_1)^2}{2 \|\mathbf{C}\|}, \quad (21)$$

$$\frac{\partial^2 A}{\partial \lambda_2 \partial \lambda_2} = \frac{\|\mathbf{g}_2\|^2 - (\hat{\mathbf{n}} \cdot \mathbf{g}_2)^2}{2 \|\mathbf{C}\|}, \quad (22)$$

$$\frac{\partial^2 A}{\partial \lambda_1 \partial \lambda_2} = \frac{\mathbf{g}_1 \cdot \mathbf{g}_2 - (\hat{\mathbf{n}} \cdot \mathbf{g}_1)(\hat{\mathbf{n}} \cdot \mathbf{g}_2)}{2 \|\mathbf{C}\|} + \frac{\hat{\mathbf{n}} \cdot (\mathbf{d}_1 \times \mathbf{d}_2)}{2}. \quad (23)$$

The off-diagonal term (23) has two parts:

- The first fraction arises from differentiating $\hat{\mathbf{n}}$ — it projects $\mathbf{g}_1$ and $\mathbf{g}_2$ perpendicular to $\hat{\mathbf{n}}$.

- The second term $\hat{\mathbf{n}} \cdot (\mathbf{d}_1 \times \mathbf{d}_2)/2$ arises from $\partial \mathbf{g}_1 / \partial \lambda_2 = \mathbf{d}_1 \times \mathbf{d}_2$ (differentiating $\mathbf{g}_1 = \mathbf{d}_1 \times \mathbf{v}$ with respect to λ_2 , since $\partial \mathbf{v} / \partial \lambda_2 = \mathbf{d}_2$).

Code: 1-inside Hessian

```
vectorized_area.py: compute_area_hessian_sparse, 1-inside branch:
g1 = np.cross(d1, v) # ∂C/∂λ1
g2 = np.cross(u, d2) # ∂C/∂λ2
d1xd2 = np.cross(d1, d2)
H_11 = (g1.dot_g1 - n.dot_g1**2) / (2.0 * normC)
H_22 = (g2.dot_g2 - n.dot_g2**2) / (2.0 * normC)
H_12 = ((g1.dot_g2 - n.dot_g1 * n.dot_g2) / normC
        + n.dot_d1xd2) / 2.0
```

4.2 2-inside case

Let $p_{\text{in}1} = \mathbf{V}[v_{\text{in}1}]$, $p_{\text{in}2} = \mathbf{V}[v_{\text{in}2}]$ (fixed mesh vertices), $p_{\text{c}1} = \mathbf{p}_1(\lambda_1)$, $p_{\text{c}2} = \mathbf{p}_2(\lambda_2)$.

4.2.1 Area value

The cell's contribution is the quadrilateral area, split into two sub-triangles:

$$A = A_1 + A_2, \quad (24)$$

where

$$A_1 = \frac{1}{2} \|(\mathbf{p}_{\text{c}1} - p_{\text{in}1}) \times (\mathbf{p}_{\text{c}2} - p_{\text{in}1})\|, \quad (25)$$

$$A_2 = \frac{1}{2} \|(\mathbf{p}_{\text{c}2} - p_{\text{in}1}) \times (p_{\text{in}2} - p_{\text{in}1})\|. \quad (26)$$

Introduce the sub-triangle vectors:

$$\mathbf{u}_1 = \mathbf{p}_{\text{c}1} - p_{\text{in}1}, \quad \mathbf{v}_1 = \mathbf{p}_{\text{c}2} - p_{\text{in}1}, \quad (27)$$

$$\mathbf{u}_2 = \mathbf{p}_{\text{c}2} - p_{\text{in}1}, \quad \mathbf{v}_2 = p_{\text{in}2} - p_{\text{in}1}. \quad (28)$$

Note that $\mathbf{v}_2$ is *constant* (both vertices are fixed).

4.2.2 Area Jacobian

VP 1 appears only in A_1 (through $p_{\text{c}1} = \mathbf{u}_1 + p_{\text{in}1}$). VP 2 appears in both A_1 and A_2 (through $p_{\text{c}2}$):

$$\frac{\partial A}{\partial \lambda_1} = \frac{\partial A_1}{\partial \lambda_1} = \frac{1}{2} \hat{\mathbf{n}}_1 \cdot (\mathbf{d}_1 \times \mathbf{v}_1), \quad (29)$$

$$\frac{\partial A}{\partial \lambda_2} = \frac{\partial A_1}{\partial \lambda_2} + \frac{\partial A_2}{\partial \lambda_2} = \frac{1}{2} \hat{\mathbf{n}}_1 \cdot (\mathbf{u}_1 \times \mathbf{d}_2) + \frac{1}{2} \hat{\mathbf{n}}_2 \cdot (\mathbf{d}_2 \times \mathbf{v}_2), \quad (30)$$

where $\hat{\mathbf{n}}_1$, $\hat{\mathbf{n}}_2$ are the unit normals of sub-triangles 1 and 2 respectively.

4.2.3 Area Hessian

Define auxiliary cross products for each sub-triangle following the same pattern as Section 4.1.3:

$$\mathbf{g}_{1,1} = \mathbf{d}_1 \times \mathbf{v}_1, \quad \mathbf{g}_{2,1} = \mathbf{u}_1 \times \mathbf{d}_2, \quad (\text{sub-triangle 1}) \quad (31)$$

$$\mathbf{g}_{2,2} = \mathbf{d}_2 \times \mathbf{v}_2. \quad (\text{sub-triangle 2}) \quad (32)$$

Sub-triangle 2 does not involve λ_1 , so its only contribution to the Hessian is to $\partial^2 A / \partial \lambda_2^2$. The full Hessian entries are:

$$\frac{\partial^2 A}{\partial \lambda_1 \partial \lambda_1} = H_{11}^{(1)}, \quad (33)$$

$$\frac{\partial^2 A}{\partial \lambda_2 \partial \lambda_2} = H_{22}^{(1)} + H_{22}^{(2)}, \quad (34)$$

$$\frac{\partial^2 A}{\partial \lambda_1 \partial \lambda_2} = H_{12}^{(1)}, \quad (35)$$

where $H_{mn}^{(j)}$ denotes the Hessian entry from sub-triangle j , computed by the formulas (21)–(23) with the appropriate sub-triangle vectors substituted. Explicitly:

$$H_{22}^{(2)} = \frac{\|\mathbf{g}_{2,2}\|^2 - (\hat{\mathbf{n}}_2 \cdot \mathbf{g}_{2,2})^2}{2 \|\mathbf{C}_2\|}, \quad (36)$$

where $\mathbf{C}_2 = \mathbf{u}_2 \times \mathbf{v}_2$ and $\|\mathbf{C}_2\| = 2A_2$.

Code: 2-inside Hessian

```
vectorized_area.py: compute_area_hessian_sparse, 2-inside branch:
H_11 = H_11_sub1
H_22 = H_22_sub1 + H_22_sub2
H_12 = H_12_sub1
```

5 Steiner / Triple-Point Contributions

A *triple point* (also called a Steiner point) arises at a mesh triangle where three or more cells meet. The boundary does not pass cleanly through two VP positions; instead the three boundary segments at the triple point are required to meet at a shared interior point $\mathbf{S}$, the *Fermat–Torricelli point* of the three VPs [1].

5.1 The Fermat–Torricelli point (forward formula)

At one triple point let the three VPs be $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$ (ordered by the convention stored in `pa.tp_vp_indices`).

Non-degenerate case (all angles $< 120^\circ$). Let

$$a = \|\mathbf{p}_2 - \mathbf{p}_3\|, \quad b = \|\mathbf{p}_1 - \mathbf{p}_3\|, \quad c = \|\mathbf{p}_1 - \mathbf{p}_2\| \quad (37)$$

be the side lengths, and let A, B, C be the angles of the triangle at $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$ respectively (via the law of cosines). The barycentric weights

$$w_i = \frac{a_i}{\sin(A_i + \pi/3)}, \quad i \in \{1, 2, 3\}, \quad (38)$$

(where $a_1 = a, a_2 = b, a_3 = c$, and A_i is the angle at vertex i) give the Fermat–Torricelli point as the weighted barycentre

$$\mathbf{S} = \frac{w_1 \mathbf{p}_1 + w_2 \mathbf{p}_2 + w_3 \mathbf{p}_3}{w_1 + w_2 + w_3}. \quad (39)$$

Degenerate case (some angle $\geq 120^\circ$). When the angle at vertex i is $\geq 2\pi/3$, the total distance $\sum_j \|\mathbf{p}_j - \mathbf{S}\|$ is minimized by placing $\mathbf{S}$ at that vertex:

$$\mathbf{S} = \mathbf{p}_k, \quad k = \arg \max_i A_i \quad \text{if } A_k \geq 2\pi/3. \quad (40)$$

Code: Steiner point

```
vectorized_steiner.py: compute_steiner_points(pa)
```

5.2 Steiner perimeter value

At one triple point with Steiner point $\mathbf{S}$, each of the three cells k (indexed by its pair of adjacent VPs $\mathbf{p}_a, \mathbf{p}_b$) contributes a perimeter value

$$P_k^{\text{tp}} = \|\mathbf{p}_a - \mathbf{S}\| + \|\mathbf{p}_b - \mathbf{S}\| - \|\mathbf{p}_a - \mathbf{p}_b\|. \quad (41)$$

The total Steiner perimeter is

$$P_{\text{st}}(\boldsymbol{\lambda}) = \sum_{(\text{tp}, \text{cell})} P_k^{\text{tp}}. \quad (42)$$

Code: Steiner perimeter value

```
vectorized_steiner.py: compute_steiner_perimeter(pa, steiner_pts)
Arrays pa.tp_contrib_vp1, pa.tp_contrib_vp2, pa.tp_contrib_tp_idx enumerate the (tp, cell)
pairs.
```

5.3 Steiner area value

Each cell k at a triple point is assigned a *void area* and a *corner area*:

$$A_{\text{void},k} = \text{area}(\mathbf{p}_a, \mathbf{p}_b, \mathbf{S}), \quad (43)$$

$$A_{\text{corner},k} = \text{area}(\mathbf{V}[v_{\text{mesh}}], \mathbf{p}_a, \mathbf{p}_b), \quad (44)$$

where $\mathbf{V}[v_{\text{mesh}}]$ is the mesh vertex of the triple-point triangle that belongs to cell k , stored in `pa.tp_contrib_mesh_vertex`. The Steiner area contribution for cell k is $A_k^{\text{tp}} = A_{\text{void},k} + A_{\text{corner},k}$.

Code: Steiner area values

```
vectorized_steiner.py: compute_steiner_areas(pa, steiner_pts)
```

5.4 Steiner derivatives: finite-difference schemes

The Steiner perimeter value (42) and area values are smooth functions of $\boldsymbol{\lambda}$; their analytical derivatives involve the implicit-function-theorem formula for $\partial \mathbf{S} / \partial \lambda_i$ (and higher-order analogues), which is algebraically complex and not yet implemented. *All Steiner derivatives are currently computed by finite differences.*

Restricted support. Only VPs in the set `pa.tp_affected_vps` can have non-zero Steiner derivatives. This set contains the (at most three) VPs at each triple point — typically $3 \times T$ entries where T is the number of triple points. All finite-difference loops iterate only over this set.

5.4.1 Perimeter gradient (forward FD)

$$\frac{\partial P_{\text{st}}}{\partial \lambda_i} \approx \frac{P_{\text{st}}(\boldsymbol{\lambda} + \varepsilon \mathbf{e}_i) - P_{\text{st}}(\boldsymbol{\lambda})}{\varepsilon}, \quad \varepsilon = 10^{-6}, \quad (45)$$

for each $i \in \text{tp_affected_vps}$. The unperturbed Steiner point `compute_steiner_points(pa)` is computed once before the loop; the perturbed value is recomputed inside the loop.

Code: Steiner gradient

```
vectorized_steiner.py: compute_steiner_perimeter_gradient(pa, eps=1e-6)
```

5.4.2 Area Jacobian (forward FD, sparse output)

$$\frac{\partial A_k^{\text{st}}}{\partial \lambda_i} \approx \frac{A_k^{\text{st}}(\boldsymbol{\lambda} + \varepsilon \mathbf{e}_i) - A_k^{\text{st}}(\boldsymbol{\lambda})}{\varepsilon}, \quad \varepsilon = 10^{-7}, \quad (46)$$

for $k = 0, \dots, N-2$ and $i \in \text{tp_affected_vps}$. The result is written directly into the sparse Jacobian values array at the pre-computed offsets `pa.nnz_lookup[k, i]`.

Code: Steiner area Jacobian

```
vectorized_steiner.py: compute_steiner_area_jacobian_sparse(pa, eps=1e-7)
```

5.4.3 Perimeter Hessian (nested central FD)

The Hessian is computed by central differences *applied to the gradient function* (45):

$$\frac{\partial^2 P_{\text{st}}}{\partial \lambda_i \partial \lambda_j} \approx \frac{g_j(\boldsymbol{\lambda} + \varepsilon_{\text{out}} \mathbf{e}_i) - g_j(\boldsymbol{\lambda} - \varepsilon_{\text{out}} \mathbf{e}_i)}{2 \varepsilon_{\text{out}}}, \quad (47)$$

where $g_j(\boldsymbol{\lambda}) := \partial P_{\text{st}} / \partial \lambda_j$ is itself evaluated by forward FD (45). The step sizes are

$$\varepsilon_{\text{out}} = 10^{-5}, \quad \varepsilon_{\text{in}} = \varepsilon_{\text{out}} \times 0.1 = 10^{-6}. \quad (48)$$

Remark 5.1 (Nested FD scale separation). The inner step ε_{in} must be strictly smaller than the outer step ε_{out} . If both steps are equal, the central-difference approximation of the outer derivative measures the second-order variation in P_{st} , but the inner gradient itself contains a bias of order ε_{in} ; when $\varepsilon_{\text{in}} = \varepsilon_{\text{out}}$ these two sources of error interfere destructively and the Hessian approximation is *first-order* accurate in ε rather than second-order. Using $\varepsilon_{\text{in}} = 0.1 \varepsilon_{\text{out}}$ separates the scales and restores second-order convergence.

Code: Steiner perimeter Hessian

```
vectorized_steiner.py: compute_steiner_perimeter_hessian_fd(pa, eps=1e-5)
eps_inner = eps * 0.1
grad_plus = compute_steiner_perimeter_gradient(pa, eps=eps_inner)
grad_minus = compute_steiner_perimeter_gradient(pa, eps=eps_inner)
hess_col_i = (grad_plus - grad_minus) / (2.0 * eps)
```

5.4.4 Area Hessian (nested forward FD)

The multiplier-weighted area Hessian is computed by forward differences applied to the area Jacobian:

$$\sum_{k=0}^{N-2} \mu_k \frac{\partial^2 A_k^{\text{st}}}{\partial \lambda_i \partial \lambda_j} \approx \sum_{k=0}^{N-2} \mu_k \frac{J_{kj}(\boldsymbol{\lambda} + \varepsilon_{\text{out}} \mathbf{e}_i) - J_{kj}(\boldsymbol{\lambda})}{\varepsilon_{\text{out}}}, \quad (49)$$

where $J_{kj}(\boldsymbol{\lambda}) := \partial A_k^{\text{st}} / \partial \lambda_j$ is evaluated by forward FD (46). The step sizes are

$$\varepsilon_{\text{out}} = 10^{-5}, \quad \varepsilon_{\text{in}} = 10^{-7}. \quad (50)$$

Here the inner step is already much smaller than the outer one (a factor of 10^2), so there is no nested-scale accuracy problem analogous to Remark 5.1.

Code: Steiner area Hessian

```
vectorized_steiner.py: compute_steiner_area_hessian_fd(pa, multipliers, eps=1e-5)
```

5.4.5 Precision limitation

Because the Steiner Hessian involves nested FD, the achievable validation tolerance is approximately 10^{-4} (absolute), compared with $\sim 10^{-10}$ for the fully analytical regular-perimeter and area Hessians. The residual error is dominated by the rounding noise in the inner gradient evaluation. Replacing the FD Steiner derivatives with the implicit-function-theorem closed-form (documented in docs/ANALYTICAL_STEINER_DERIVATIVES_PLAN.md) would recover machine-precision accuracy; this is planned but not yet implemented.

6 Lagrangian Hessian Assembly

6.1 The Lagrangian

IPOPT minimizes the augmented objective by maintaining the Lagrangian

$$\mathcal{L}(\boldsymbol{\lambda}, \boldsymbol{\mu}) = P(\boldsymbol{\lambda}) + \sum_{k=0}^{N-2} \mu_k (A_k(\boldsymbol{\lambda}) - \bar{A}), \quad (51)$$

where μ_k are Lagrange multipliers supplied by IPOPT at each iteration.

6.2 Hessian assembly

IPOPT's Hessian callback receives the current $\boldsymbol{\lambda}$, the multipliers μ_k , and an objective scaling factor σ (used when the objective is rescaled internally). The callback must return

$$H = \sigma \nabla^2 P_{\text{reg}} + \sigma \nabla^2 P_{\text{st}} + \sum_{k=0}^{N-2} \mu_k \nabla^2 A_k^{\text{reg}} + \sum_{k=0}^{N-2} \mu_k \nabla^2 A_k^{\text{st}}. \quad (52)$$

Each of the four terms is computed by a separate function and then summed in the flat sparse-values array:

Term	Function	Method
$\nabla^2 P_{\text{reg}}$	<code>compute_perimeter_hessian_sparse</code>	analytical (§3.3)
$\nabla^2 P_{\text{st}}$	<code>compute_steiner_perimeter_hessian_fd</code>	nested central FD (§5.4.3)
$\sum_k \mu_k \nabla^2 A_k^{\text{reg}}$	<code>compute_area_hessian_sparse</code>	analytical (§4.1.3, §4.2)
$\sum_k \mu_k \nabla^2 A_k^{\text{st}}$	<code>compute_steiner_area_hessian_fd</code>	nested forward FD (§5.4.4)

Code: Hessian callback

```
perimeter_optimizer.py: IPOPTProblemAdapter.hessian()
return (obj_factor * (perim_hess + steiner_perim_hess)
        + area_hess + steiner_area_hess)
```

6.3 Lower-triangle convention

IPOPT requires only the *lower triangle* of the symmetric matrix H , i.e. entries (i, j) with $i \geq j$. All Hessian functions return a flat array of values at positions `(pa.hess_row, pa.hess_col)` satisfying `hess_row[k] >= hess_col[k]` for all k .

6.4 Sparsity structure

The non-zero pattern of H (lower triangle) is the union of contributions from three sources, determined at compile time in `PartitionContour.compile_arrays()`:

1. **Perimeter segments.** Each segment (a, b) contributes the three entries (a, a) , (b, b) , $(\max(a, b), \min(a, b))$. Source: `pa.seg_vp1`, `pa.seg_vp2`.
2. **Boundary triangles.** Each triangle with VPs (vp_1, vp_2) contributes the same three entries. Because the same pair of VPs defines both the perimeter segment and the bounding triangle, the perimeter and area patterns are identical — no new entries are added. Source: `pa.btri_vp1`, `pa.btri_vp2`.
3. **Triple points.** Each triple point has three VPs, giving up to six off-diagonal entries (all pairs within the triple point) plus three diagonal entries. Source: `pa.tp_vp_indices`.

Summary table

Quantity	Regular	Steiner / triple-point
Perimeter value P	Analytical (7)	Analytical (42)
Perimeter gradient ∇P	Analytical (11)	Forward FD (45), $\varepsilon = 10^{-6}$
Area Jacobian J	Analytical (19), (29)–(30)	Forward FD (46), $\varepsilon = 10^{-7}$
Perimeter Hessian $\nabla^2 P$	Analytical (13)–(15)	Nested central FD (47), $\varepsilon_{\text{out}} = 10^{-5}$, $\varepsilon_{\text{in}} = 10^{-6}$
Area Hessian $\nabla^2 A_k$	Analytical (21)–(23), (33)–(36)	Nested forward FD (49), $\varepsilon_{\text{out}} = 10^{-5}$, $\varepsilon_{\text{in}} = 10^{-7}$

References

- [1] Harold W. Kuhn. Steiner’s problem revisited. pages 52–70, 1974. Background reference for the Fermat–Torricelli construction and the barycentric-weight formula for the Steiner point.
- [2] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.